

Tuần 1: Giới thiệu Python cho Security & Lập trình Socket cơ bản / Week 1: Introduction to Python for Security & Basic Socket Programming

Mục Tiêu / Objectives

Trong tuần học đầu tiên này, chúng ta sẽ đặt nền móng vững chắc cho toàn bộ khóa học bằng cách tìm hiểu vai trò của Python trong lĩnh vực An ninh mạng, đồng thời nắm bắt các nguyên lý cốt lõi của lập trình mạng thông qua Socket. Các bài thực hành code sẽ được thiết kế tăng dần từ cơ bản đến phức tạp. / In this first week, we will lay a solid foundation for the entire course by exploring the role of Python in Cybersecurity and grasping the core principles of network programming through Sockets. Hands-on coding will progress from basic to complex.

Mục tiêu cụ thể / Specific Objectives:

1. Hiểu rõ tại sao Python là "vũ khí" số 1 của các hacker và chuyên gia bảo mật.
2. Nắm vững khái niệm Địa chỉ IP, Port (Cổng), TCP/UDP và Localhost.
3. Thực hành lập trình Socket qua 3 cấp độ: Cơ bản (Echo) -> Trung bình (Chat vòng lặp) -> Phức tạp (Bảo mật & Quản lý lỗi).
4. Khắc sâu nguyên tắc Đạo đức Hacker (White Hat Ethics) và tuyệt đối chỉ thực hành trên Localhost (127.0.0.1).

Lý Thuyết / Theory (with definitions and examples)

1. Tại sao Python lại thống trị lĩnh vực An ninh mạng?

- **Cú pháp rõ ràng:** Giúp chuyên gia đọc hiểu mã độc (malware) nhanh chóng.
- **Thư viện khổng lồ:** `socket` (mạng cơ bản), `Scapy` (phân tích gói tin), `Requests` (web hacking), `Cryptography` (mã hoá).
- **Đa nền tảng:** Chạy mượt mà trên Kali Linux, macOS, và Windows.

2. Khái niệm Mạng Cơ Bản

- **IP Address & Port:** Nếu IP (vd: `192.168.1.5`) là địa chỉ tòa nhà, thì Port (vd: `80`, `443`, `9999`) là số phòng.
- **Localhost (127.0.0.1):** Địa chỉ Loopback. Gửi dữ liệu tới địa chỉ này có nghĩa là "gửi cho chính mình". Đây là môi trường cách ly (Sandbox) an toàn nhất để học bảo mật.
- **TCP vs UDP:** TCP giống như gọi điện thoại (bắt tay 3 bước, đảm bảo an toàn, không rớt gói). UDP giống như gửi thư (gửi đi không cần biết bên kia nhận được chưa, tốc độ cao nhưng thiếu tin cậy).

3. Socket là gì?

- Socket là điểm cuối (endpoint) để hai phần mềm nói chuyện với nhau.
- **Máy chủ (Server):** Tạo socket -> **bind** (gắn IP/Port) -> **listen** (nghe) -> **accept** (chấp nhận kết nối).
- **Máy khách (Client):** Tạo socket -> **connect** (gọi điện tới Server).

Cảnh Báo An Toàn & Đạo Đức / Safety Warnings and Ethical Notices

[!WARNING]

CẢNH BÁO PHÁP LÝ & ĐẠO ĐỨC (LEGAL & ETHICAL WARNING):

1. Việc cố ý quét hoặc kết nối đến hệ thống của người khác khi chưa được phép là **hành vi vi phạm pháp luật**.
2. Khóa học này chỉ cho phép thực hành trên **localhost (127.0.0.1)**. Bất kỳ bài nộp nào sử dụng IP công cộng hoặc IP LAN (ví dụ 192.168.x.x) để tấn công/kết nối trái phép đều bị điểm 0.

Thực Hành Code / Hands-On (Từ Cơ Bản Đến Phức Tạp)

Chúng ta sẽ trải qua 3 cấp độ code. Ở mỗi cấp độ, bạn hãy tạo các file Python mới, mở 2 cửa sổ Terminal (1 cho Server, 1 cho Client) và chạy thử nghiệm.

Cấp độ 1: Giao tiếp Cơ bản (Basic Echo Server)

Mục tiêu: Viết đoạn code ngắn nhất có thể để kết nối thành công. Server nhận 1 tin nhắn và in ra màn hình.

1. **basic_server.py** (Mở port và nghe)

```
import socket # Khai tạo Socket (IPv4, TCP) server = socket.socket(socket.AF_INET, socket.SOCK_STREAM) # Gắn vào cổng 9999 trên máy của mình (Localhost) server.bind(('127.0.0.1', 9999)) # Bật chuông nghe (tôi sẽ chờ 1 kết nối) server.listen(1) print("Server đang chờ kết nối trên port 9999...") # Chấp nhận khi có client gửi tới (Chương trình sẽ dừng ở đây chờ) client, address = server.accept() print(f"Có người kết nối tới: {address}") # Nhận tin nhắn (tôi sẽ 1024 bytes) msg = client.recv(1024).decode('utf-8') print(f"Tin nhắn nhận được: {msg}") # Đóng kết nối client.close() server.close()
```

2. **basic_client.py** (Kết nối và gửi)

```
import socket client = socket.socket(socket.AF_INET, socket.SOCK_STREAM) # Gửi tín hiệu tới Server client.connect(('127.0.0.1', 9999)) # Gửi tin nhắn (phải encode sang bytes) client.send("Xin chào, tôi là Client!".encode('utf-8')) client.close()
```

(Chạy server trước, sau đó chạy client. Cả hai sẽ kết thúc ngay sau khi gửi/nhận 1 tin nhắn).

Cấp độ 2: Chat Liên tục (Continuous Chat)

Mục tiêu: Đưa tính năng nhận/gửi vào vòng lặp `while True` để có thể chat qua lại nhiều lần. Thêm lệnh thoát (`EXIT`).

1. `chat_server.py`

```
import socket
server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1) # Giúp ch...
l...i code không b... l...i "Port in use"
server.bind(('127.0.0.1', 9999))
server.listen(1)
print("Server Chat đang ch...")
client, address = server.accept()
print(f"Đã kết nối với {address}")
while True: # Ch...
    nh...n tin nh...n data = client.recv(1024).decode('utf-8')
    if not data or data == 'EXIT':
        print("Client đã ngắt kết nối.")
        break
    print(f"Client: {data}")
    # Server nh...p câu tr... l...i reply = input("Server tr... l...i: ")
    client.send(reply.encode('utf-8'))
client.close()
server.close()
```

2. `chat_client.py`

```
import socket
client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect(('127.0.0.1', 9999))
while True:
    msg = input("Nh...p tin nh...n (gõ EXIT để thoát): ")
    client.send(msg.encode('utf-8'))
    if msg == 'EXIT':
        break
    # Ch... Server ph...n h...i reply =
    client.recv(1024).decode('utf-8')
    print(f"Server nói: {reply}")
client.close()
```

Cấp độ 3: Máy chủ Bảo mật & Quản lý Lỗi (Secure Server)

Mục tiêu: Trong thực tế, Server phải chạy 24/7, không được phép "crash" khi có lỗi, và phải từ chối các kết nối độc hại. Chúng ta sẽ dùng `try...except` và thư viện `logging`.

`secure_server.py`

```
import socket
import logging
# Dùng Logging thay cho print để lưu vết
chuyên nghiệp
logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(message)s')
def run_secure_server():
    # Context manager 'with' để
    đóng socket khi x...y ra s... c... with
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    server.bind(('127.0.0.1', 9999))
    server.listen(5)
    logging.info("Secure Server đang lắng nghe trên 127.0.0.1:9999")
    while True:
        try:
            client_conn, client_addr = server.accept()
            # B...O M...T:
            Ch...n các kết nối không an toàn từ Localhost
            if client_addr[0] != '127.0.0.1':
                logging.warning(f"Phát hiện IP lạ {client_addr[0]}! Đang ch...n...")
                client_conn.close()
                continue
            with client_conn:
                logging.info(f"Đã kết nối với máy khách an toàn: {client_addr}")
                # Quá trình giao tiếp
                while True:
                    data = client_conn.recv(1024)
                    if not data:
                        break
                    msg = data.decode('utf-8')
                    logging.info(f"Nh...n từ client: {msg}")
                    # Ph...n h...i response = f"[Server Ack] đã nh...n {len(msg)} ký t..."
                    client_conn.sendall(response.encode('utf-8'))
        except KeyboardInterrupt:
```

```
logging.info("■ Admin ■ă ch■ ■ng t■t Server.") break except Exception as e: logging.error(f"■ L■i h■ thng: {e}") if __name__ == "__main__": run_secure_server()
```

Ghi chú: Bạn có thể dùng `chat_client.py` ở Cấp độ 2 để kết nối thử vào Secure Server Cấp độ 3 này.

Bài Về Nhà / Homework

Đề bài: Ứng dụng mã hoá tin nhắn sơ cấp (Caesar Cipher)

Dựa trên kiến thức của Cấp độ 2 (Chat liên tục), hãy nâng cấp ứng dụng Chat của bạn:

1. **Client:** Viết một hàm dịch vòng chữ cái (Caesar Cipher) đơn giản. Khi người dùng nhập "HELLO", mã hoá nó thành "KHOOR" (dịch 3 ký tự) rồi mới gửi đi qua mạng.
2. **Server:** Khi nhận được "KHOOR", nó phải gọi hàm giải mã ngược lại 3 ký tự để in ra màn hình từ "HELLO".

Yêu cầu nộp bài:

Nén 2 file `crypto_client.py` và `crypto_server.py` kèm theo ảnh chụp màn hình terminal minh chứng.

Đánh Giá / Assessment Rubric Table

Tiêu chí / Criteria	Xuất sắc / Excellent (90-100%)	Tốt / Good (70-89%)	Cần cố gắng / Needs Improvement (<70%)
1. Tuân thủ An toàn	Hardcode <code>127.0.0.1</code> . Tuyệt đối không để hở IP ra LAN/Public. (30 điểm)	Dùng localhost nhưng code viết thiếu cẩn thận, dễ nhầm lẫn. (20 điểm)	Dùng IP <code>0.0.0.0</code> . (0 điểm, FAIL toàn phần).
2. Logic Mã Hoá	Mã hoá và giải mã chính xác 2 chiều, xử lý tốt khoảng trắng (space). (40 điểm)	Mã hoá được nhưng thỉnh thoảng lỗi ký tự đặc biệt. (25 điểm)	Mã hoá sai nguyên lý hoặc không chạy được. (10 điểm)

3. Xử Lý Vòng Lặp	Client và Server chat qua lại liên tục không bị kẹt (block). Thoát duyên dáng khi gõ EXIT. (30 điểm)	Có vòng lặp nhưng lỗi logic khiến ứng dụng bị treo giữa chừng. (15 điểm)	Chỉ gửi được 1 tin nhắn rồi ngắt kết nối. (0 điểm)
--------------------------	--	--	--